

EduBot 多 Agent 路由与任务编排

从“每轮选 Agent”升级为可持续、可暂停恢复、可审计的多轮任务控制面

公司：阿里巴巴 | 角色：核心开发 / 链路审计【具体岗位待确认】 | 状态：一期已上线 + 目标架构设计

30 秒版本 我做的是教育大模型的多 Agent 控制链。核心不是把 Query 分成 Main 或 SubAgent，而是让专业任务在多轮里有稳定身份、能被抢断和恢复，并在并发、流式输出和异常情况下保持状态可解释。我参与收敛 Gate、Router、Task/Topic、ToolCall/Handback 链路，并用生产日志完成 163 个多轮 Session、902 请求的全链路审计，定位恢复判定和 LGI 单调性问题。

一、项目背景与业务问题

Main Agent 适合开放问答、澄清和能力协调，SubAgent 适合持续讲解、辅导或练习。如果每轮都让 Main 重新选择，时延与成本上升且专业任务容易摇摆；如果只用 active_agent 粘住，又无法表达换题、暂停、恢复、完成和同 Agent 多任务。因此需要一个独立控制面管理“本轮归谁”和“跨轮任务如何变化”。

方案	优势	核心缺陷
每轮 Main 重新选择	灵活	重复模型成本；相邻轮抖动；进度难恢复
只记 active_agent	低时延	无法区分任务身份；易错误黏住；不能暂停/恢复
Router + Task 生命周期	连续、可恢复、可审计	需要状态机、并发控制和协议闭合

二、我的工作与可交付结果

- 参与统一请求链：把 Scene Direct、Main ToolCall Handoff、active sticky、SubAgent Handback 统一收敛到 Router 循环。
- 参与 Handoff Gate 的规则优先与单 token 小模型方案，明确 advice、apply 与 final target 三层事实。
- 围绕 Session / Turn / Topic / Task / Owner / LGI / revision 建立状态语义，覆盖 CREATE、CONTINUE、PREEMPT、RESUME、COMPLETE。

- 4. 梳理 ToolCall-ToolResult 闭合、Chat/Live/SSE 输出、异常 healing、CAS 与迟到结果等工程边界。
- 5. 基于生产 SLS 做 on 桶全量链路对账、分支定向抽样与人工语义复核，定位 P0/P1 问题并给出可落地方案。

不要说过头 一期已经具备 active/paused 两个互斥槽位、关键转换 CAS 和统一 Router；Task Registry、多 paused、真实 checkpoint/restore、command ledger 与 transactional outbox 是目标设计或算法 Demo 合同，不能表述为已完整上线。

三、业务链路

用户 Query

-> 入口解析 / 安全审核 / 确定性干预

-> PromptGet + Handoff Gate + Proactive Gate 并行准备

-> Router 校验实验开关、revision 与最新 Task 状态

-> MAIN | SCENE_DIRECT | ACTIVE_HANDOFF | HANDOFF_INTENT

-> Main / SubAgent 流式执行

-> ToolCall / Final / Handback / Error 再进入 Router

-> SSE 交付 + 消息/状态持久化 + 观测闭环

五种任务动作

动作	触发	状态变化	LGI 规则
CREATE	Scene 首轮或 Main 新建 Handoff	Main -> 新 active；清旧 paused	新任务从初始值开始
CONTINUE	当前 Query 仍属 active Task	保持 Task/Topic/Owner	吸收合法 ACK
PREEMPT	换题或明确暂离	active -> paused；本轮 Main	先保存最新确认值
RESUME	明确恢复且目标唯一	paused -> active	恢复 paused 保存值
COMPLETE	SubAgent Handback	清任务并交回 Main	完成后不可恢复

四、核心算法设计

4.1 规则优先 + active continuity 小模型

- 6. 高精度规则先处理明确继续、形式调整、系统控制、暂停/退出、情绪支持等后果清晰的表达。

7. 有 active 且规则未命中时，小模型只输出 0/1，不生成业务答案；输入仅保留当前 Query、上一轮用户话术、任务目标、Topic、最近进度和 Agent 能力。
8. 用 $\text{margin} = \log P(\text{Main}) - \log P(\text{SubAgent})$ 做双阈值：明显继续、明显回 Main、中间不确定区。配置示例为 ≤ -0.5 继续， ≥ 2.875 抢断，中间 CLARIFY。
9. 异常时如果已确认合法 active，优先 CONTINUE_ACTIVE；状态本身读失败则不做写操作。这里的 fail-open 是用户连续性开放，观测必须标记 fallback。

4.2 paused-only 恢复与为什么后来发现它有问题

一期只有一个 paused 候选，恢复规则尝试综合 Task ID、目标/摘要关键词、Agent 别名和 resume cue。设计初衷是降低误恢复，但线上审计表明纯词面规则既会在“继续聊/接着告别”等语境中误召，又会漏掉真正的短句“继续”和同主题内容追问。更合理的方向是规则召回 + 语义复核，并用 Query 与 paused goal/topic 的匹配分数选择目标。

五、工程设计

5.1 权责与状态一致性

10. Gate 只产 advice；Router 校验实验配置、revision 和目标 Task 后才提交 effect；Task Store 不理解语义。
11. 请求级配置做快照，避免 Gate、Router、Main 在同一请求里读取不同版本。
12. 关键 PREEMPT/RESUME 使用 revision + CAS，复合状态在 Redis 事务中写入；dispatch 前再次确认 active Task。
13. 所有终态应绑定 session_id + task_instance_id + command_id + expected_version，避免同 Agent 的旧结果污染新 Task。

5.2 协议闭合与流式交付

14. Main 交给 SubAgent 是 ToolCall；任务完成或抢断回 Main 时必须补同 tool_call_id 的 ToolResult，且新 Query 位于闭合结果之后。
15. active 期间 ToolCall 暂时 orphan 是合法开放协议；Handback、Preempt、过期 healing 时必须闭合。
16. Task LGI 是任务续传位置，Session 高水位保证全会话帧单调；两者都不是 Agent checkpoint。

17. Chat SSE、Live SSE 与 JSON 共用 Router，但 final/error/handoff 的外部表达不同，需要分别验收。

5.3 CAS 还不够

CAS 防止旧快照覆盖新状态，但不能防重复命令、状态已提交而 dispatch 未发生、SSE 已发而消息未落盘、同 Session 双流和跨 Worker 主动候选重复消费。目标方案是 command ledger + request owner/epoch + checkpoint/restore + transactional outbox + 下游幂等。

六、线上审计与实验结果

证据	结果	面试解释
链路全量核验	163 多轮 Session / 902 请求； 898 正常终态	证明控制链可对账，不等于语义全对
决策	580 次成功：模型 329 + 规则 251	规则路径也是 Gate 成功，不能漏算
并发	47/163 Session 有重叠；相邻对 重叠 11.4%	并发是常态；CAS conflict 需重评
恢复质量	规则触发精确 0/4；显式 “继续” 召回 0/4	样本小但机制缺陷明确；需语义复核
LGI	5 个 Session、6 次非零 -> 0	场景入口重发导致；服务端需高水位保护
单日 A/B	转讲万物 +14.9pp；退出 - 4.7pp；放弃 -2.5pp	1128/197 Sessions，真实干预率 6.7%，只作初步证据

指标口径 实验收益必须同时说出窗口、样本量和真实干预率。线上审计的 18 个语义样本是分支定向抽样，不能把 PASS/PARTIAL/FAIL 分布外推为总体比例。

七、关键问题、复盘与下一步

问题	根因	改进
恢复规则假阳性/假阴性	词面 cue 缺少当前活动语境与目标匹配	规则召回后模型复核；goal/topic 与 Query 相似度；候选消歧
LGI 回退	incoming=0 直接覆盖非零 checkpoint	max(existing,incoming) 或 CAS 单调；重入单独

问题	根因	改进
		建模
CAS conflict 无重评	正确建议和错误建议都可能被随机丢弃	按新 revision 重跑低成本规则；记录冲突轮终态
任务碎片化	CLARIFY 与 Main 工具反复新建/恢复	统一 CLARIFY 语义；同 Topic 恢复与新建边界
状态/dispatch 分裂	跨系统无事务	next state + command result + event + outbox 同事务

八、面试高频问答

Q1. 为什么这不是普通意图分类？

建议回答：意图分类只回答当前文本像什么；本项目还要结合已有任务、状态版本和恢复资格决定本轮 owner，并维护跨轮 Task 生命周期、协议与流式进度。

18. 追问展开：用 ‘换题—临时问答—恢复—完成’ 四轮例子解释状态变化。

Q2. Gate 和 Router 为什么分开？

建议回答：Gate 是可替换的算法建议层，Router 是控制权威。分开后可以 shadow、分桶、校准模型而不直接改状态，也能由 Router 做开关、revision、CAS 和最终兜底。

19. 追问展开：Gate hit 不能当业务成功，至少还要看 apply、transition、final target 和用户可见结果。

Q3. 为什么规则在模型前？

建议回答：明确暂停/继续等高精度控制意图不需要模型；规则更快、可解释，在历史或模型异常时仍可工作。模型只处理语义模糊区。

20. 追问展开：规则不能无限扩张，恢复规则的线上失准正说明复杂语义需要模型参与。

Q4. 为什么用 logprob margin 而不是最大概率？

建议回答：margin 直接表达 Main 与 SubAgent 两个动作的相对证据，并支持不对称双阈值：自动继续和自动抢断的风险不同，中间区交 Main 澄清。

21. 追问展开：阈值必须按场景与误判成本校准，不是固定常数。

Q5. CLARIFY 为什么难？

建议回答：它既可以是瞬时让 Main 问一句，也可以被实现成 durable pause。两种语义会影响任务是否可恢复和是否产生碎片，必须先冻结产品合同再编码状态机。

Q6. 为什么 active 和 paused 不能同时存在？

建议回答：这是一期容量简化，不是理论要求。它降低状态组合和恢复消歧成本，但新任务会覆盖旧 paused 的恢复资格。审计显示多数回归指向最近任务，容量暂时可用，短板主要在判定。

Q7. Topic 和 Task 有什么区别？

建议回答：Topic 是语义上下文边界，Task 是某个 Agent 对该目标的一次执行。一个 Topic 最终可以关联多次 Task；一期 `topic_id==task_id` 只是迁移简化。

Q8. LGI、progress summary、checkpoint 有什么区别？

建议回答：LGI 回答客户端播放到哪里，summary 回答用户看到了什么，checkpoint 回答 Agent 内部执行到哪个可恢复安全点。只有 checkpoint 能真正恢复内部工作流。

Q9. 为什么 CAS 不够？

建议回答：CAS 只保护基于旧版本的写，不解决相同 command 重放、状态与 dispatch 原子性、迟到结果、双流和 SSE/持久化分叉，所以还需要幂等台账、owner/epoch、outbox 和 settled。

Q10. ToolCall 为什么必须闭合？

建议回答：Main 的模型历史要求每个 ToolCall 有配对 ToolResult，否则会认为工具仍未完成，导致重复调用或上下文非法。闭合还必须与最新 Query 保持正确顺序。

Q11. 线上审计怎么做？

建议回答：先按 `matched.edu_main_gate` 精确取 on 请求，再以 `session_id+request_id` 关联 Gate、apply、dispatch、transition、终态六类标记做全量对账；异常分支定向抽样，合成整段时间线做语义复核。

22. 追问展开：说明分支抽样不能外推总体比例。

Q12. 你发现的最关键问题是什么？

建议回答：暂停恢复规则双向失准和 LGI 非零回退。前者导致任务误拉回或长期悬置，后者破坏续播单调性；两者都通过全量链路和语义样本相互验证。

Q13. 如果重做恢复，你会怎么设计？

建议回答：先召回合法 paused 候选，再用 Query、goal、topic、recent progress 做语义匹配；规则只处理显式 Task ID 和高精度 cue；低置信度交 Main 澄清，并记录候选、评分、apply 与实际恢复效果。

Q14. 怎么降低首 token 时延？

建议回答：并行 PromptGet/Gate/Proactive，规则前置，active sticky 绕过 Main，Gate 单 token+短 deadline+连接池；但只取消最终不需要的分支，不能把投机 hint 当已提交 route。

Q15. 目标架构为什么需要 outbox？

建议回答：状态提交和跨服务 dispatch 无法放进同一个远程事务。把 dispatch record 与 next state 同事务写入，再由 worker 至少一次投递、下游按 command_id 幂等，覆盖进程中途崩溃窗口。

九、两分钟项目陈述模板

项目面向教育大模型的多轮专业任务承接。早期方案要么每轮让 Main 重新选择 Agent，成本高且不连续；要么只记 active_agent，换题后容易粘住，也无法安全暂停恢复。我们把它抽象成 Gate + Router + Task 状态链：Gate 用高精度规则和单 token 小模型给出继续、抢断、恢复或澄清建议；Router 结合请求级实验配置和 revision 做最终决策，并维护 active/paused Task、Topic、Owner 和 LGI。Main 通过 ToolCall 发起 Handoff，SubAgent 通过 Handback 结束任务，Router 补 ToolResult 形成闭环。工程上关键是把 advice 和 effect 分开、用 CAS 防旧快照覆盖，并分别处理 Chat/Live/SSE 交付。我还做了生产日志审计：对 163 个多轮 Session、902 请求完成全链路对账，终态 898 个正常，同时发现暂停恢复规则精确率和召回都存在机制性问题、5 个 Session 出现 LGI 回退。对应方案是让恢复引入目标语义匹配、对 LGI 做高水位保护，并在 CAS conflict 后按新版本低成本重评。

资料依据与表达边界

边界 “当前一期” “算法 Demo” “目标形态” 须分开陈述；线上审计结论可以说，未落地的 Task Registry、checkpoint/outbox 只能说设计或演进方案。

主要依据：

- 23. 阿里实习材料/01-路由与 Handoff 完整业务逻辑.md
- 24. 阿里实习材料/02-任务编排与 Topic 完整设计.md
- 25. 阿里实习材料/04-统一架构与链路工程设计.md
- 26. 阿里实习材料/edubot-on-bucket-audit-2026-08-26.md
- 27. 阿里实习材料/各类实验收益.txt